

★ WEEK 9 · PART 1 of 3

The Kalman Filter: Predict, Then Update

The Recursion That Flew to the Moon

The predict step · the update step · the innovation · one cycle per observation

Week 8 gave the state space model and the intuition of a predict-update loop. Now we make it precise: the two-step recursion that estimates the hidden state optimally, in real time.

1

PART 1 · PREDICT & UPDATE

Two Steps, Repeated Forever

The predict step that moves the state forward, the update step that corrects it with new data, and the innovation — the surprise — that drives every correction.

The Filter That Flew to the Moon — Part 1

From a doubted paper to Apollo navigation

In 1960, Rudolf Kalman published a recursive solution to estimating a hidden state from noisy measurements. Parts of the field were skeptical — until NASA saw what it could do.

The idea

- A recursive filter: update an estimate as each measurement arrives.
- No need to re-solve over the whole history each time.
- Elegant, fast, and exactly suited to real-time systems.

The turning point

- Stanley Schmidt at NASA Ames grasped its importance.
- He adapted it for the Apollo guidance computer.
- It estimated spacecraft position and velocity from noisy sensors.

The Filter That Flew to the Moon — Part 2

Why it worked, and the lesson

The filter fused an imperfect motion model with noisy measurements to produce the best possible estimate — with limited 1960s computing. The same recursion now runs in finance.

Why it worked

- It weighted the model and the data by their uncertainties.
- It carried a running estimate of how sure it was.
- It was recursive — cheap enough to run onboard, live.

The lesson for us

- The same problem is everywhere: a hidden state, noisy data.
- Trend, volatility, and hedge ratios are hidden states.
- One recursion estimates them all, optimally and in real time.

We Have the Model — Now We Need the State

Where Week 8 left off

1

Week 8 gave the framework

State space: a hidden state evolving over time, seen only through noisy observations. We even sketched a predict-update loop.

2

But we never wrote the optimal recipe

How exactly do you move the state forward, and how much do you correct it when data arrives? Week 8 gave intuition; Week 9 gives the equations.

3

The Kalman filter is that recipe

It is the optimal recursive estimator for a linear-Gaussian state space model — the precise engine behind everything in Part II.

Part 1 in one line: two steps — predict then update — repeated once per observation.

The Big Idea: Predict, Then Update

Two steps, repeated for every observation

The filter alternates two moves. PREDICT: use the model to guess the next state, becoming less certain. UPDATE: use the new observation to correct that guess, becoming more certain.

Predict (time update)

- Push the state forward by its own dynamics.
- Uncertainty grows — you know a little less.
- This gives a prediction of the next observation.

Update (measurement update)

- Compare the prediction to the actual data.
- Correct the state toward the data.
- Uncertainty shrinks — the observation informed you.

What the Filter Carries Forward

Two things, and only two

carry forward: $(\hat{x}_{t-1|t-1}, P_{t-1|t-1}) =$ state estimate and its covariance

The estimate

- The current best guess of the hidden state.
- It summarizes everything seen so far.
- It is all you need to move to the next step.

The covariance

- How uncertain that estimate is.
- It grows in predict, shrinks in update.
- It is what lets the filter weigh model against data.

The Predict Step (Time Update)

Move the state and its uncertainty forward

$$\text{Predict: } \hat{x}_{t|t-1} = T \hat{x}_{t-1|t-1}, \quad P_{t|t-1} = T P_{t-1|t-1} T' + Q$$

The state moves

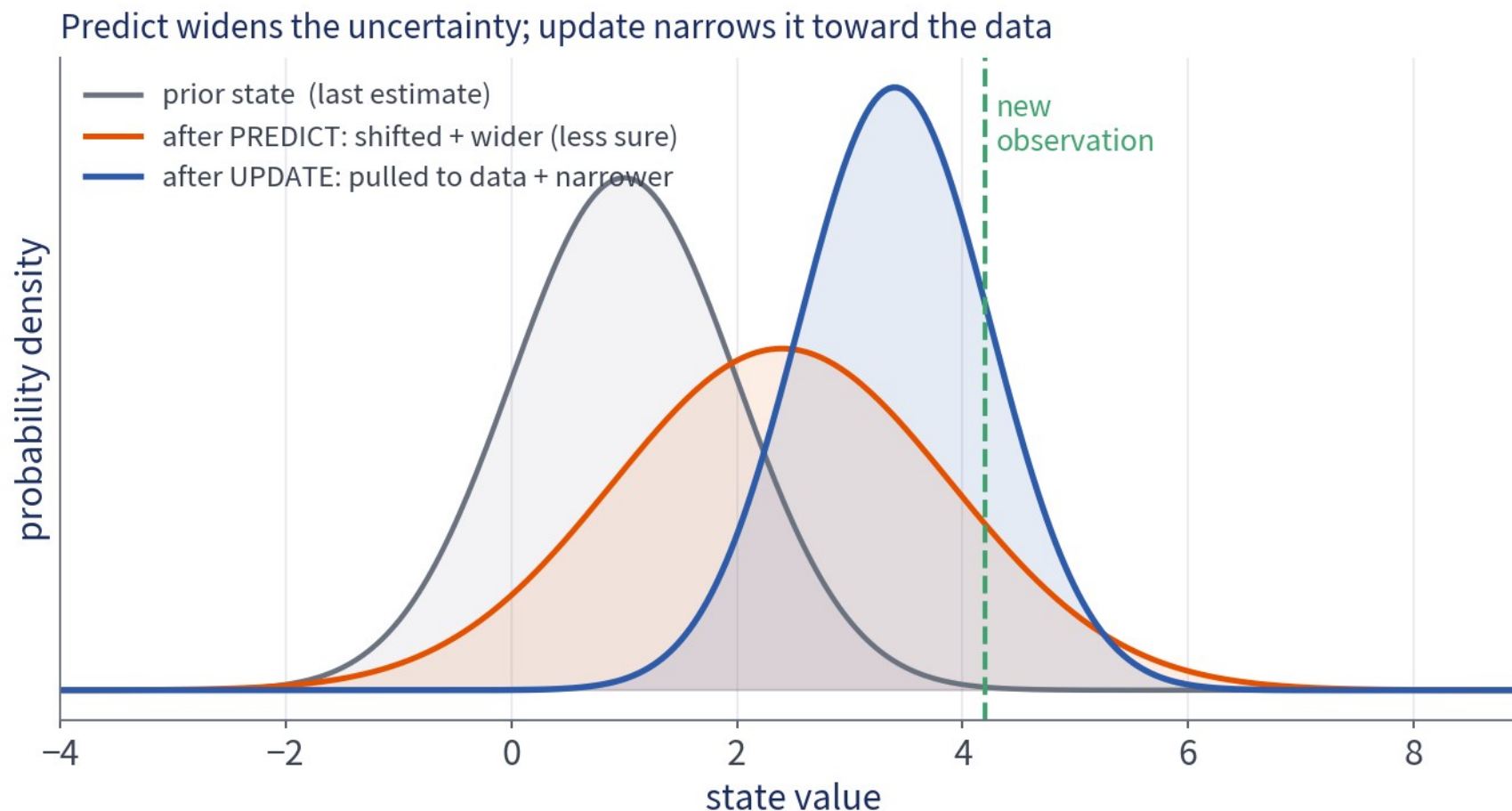
- Apply the transition T to the last estimate.
- The state coasts forward by its own dynamics.
- No data is used yet — this is a pure forecast.

The uncertainty grows

- The covariance is pushed through T and inflated by Q .
- Q is the process noise — the model isn't perfect.
- So you become LESS certain until data corrects you.

Predict Widens, Update Narrows

As a distribution over the state: predicting shifts it forward and widens it (less sure); then the new observation pulls it over and sharpens it (more sure). One full cycle.



Predicting the Next Observation

What we expect to see, and how unsure we are

$$\text{predicted obs } \hat{y}_t = Z \hat{x}_{t|t-1}; \quad \text{innovation variance } F_t = Z P_{t|t-1} Z' + H$$

The predicted observation

- Map the predicted state through Z to expect a data value.
- This is what the model says should happen next.
- Comparing it to reality is the whole point.

Its uncertainty F

- F combines state uncertainty and measurement noise H .
- It says how surprised we are ALLOWED to be.
- It scales the correction in the update step.

The Innovation: The Surprise

The signal that drives every correction

$$\text{Innovation (surprise): } v_t = y_t - \hat{y}_t = y_t - Z \hat{x}_{t|t-1}$$

What it is

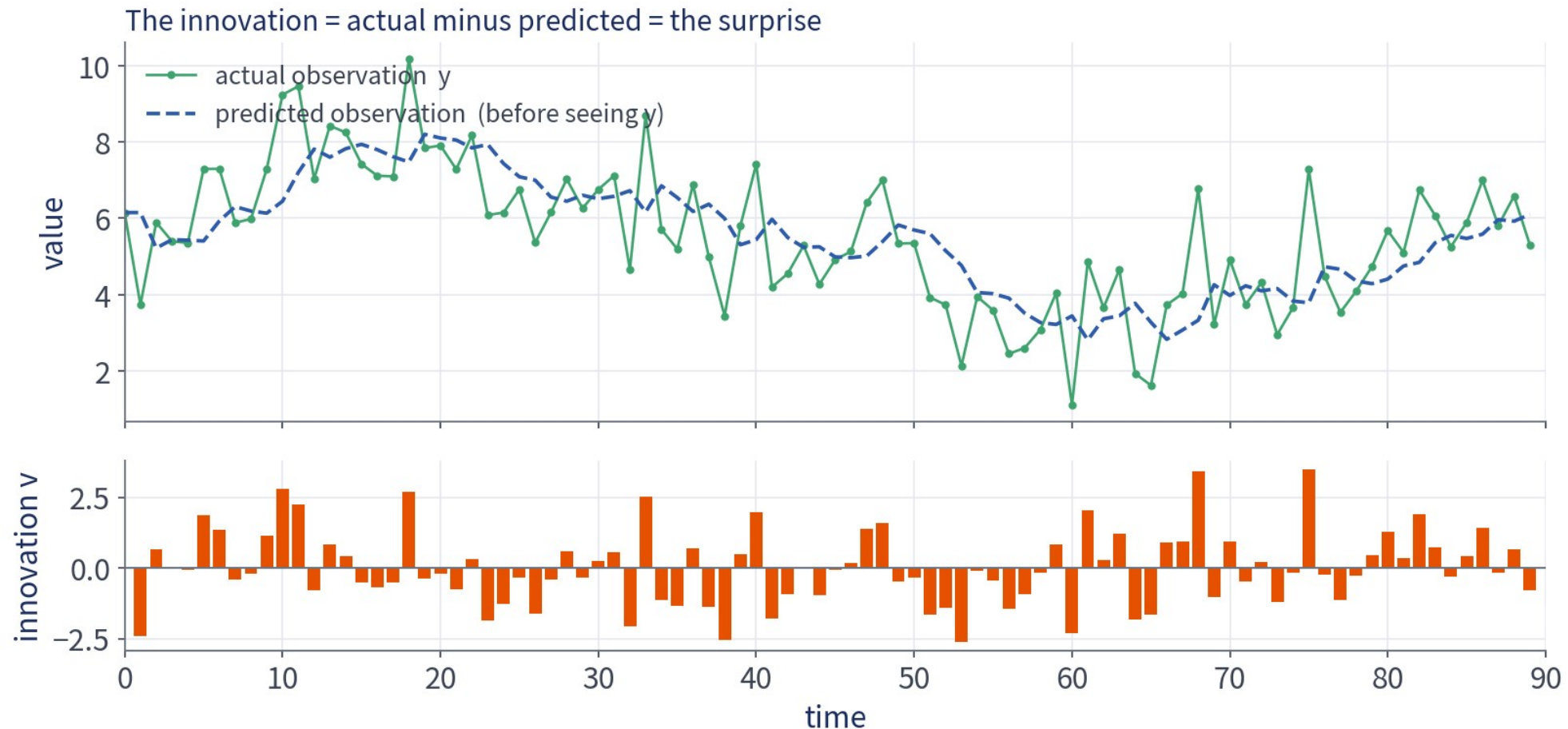
- The gap between the actual and predicted observation.
- How wrong the model's forecast turned out to be.
- The only NEW information the observation carries.

Why it is central

- The correction is built entirely from the innovation.
- No surprise $\rightarrow$ no correction needed.
- In a good model, the innovations are just white noise.

The Surprise, Each Step

Before seeing each point, the filter predicts it (dashed). The gap to the actual value (top) is the innovation (bottom) — the surprise that the update step will act on.



The Update Step (Measurement Update)

Correct the prediction with the new data

Update: $\hat{x}_{t|t} = \hat{x}_{t|t-1} + K_t v_t$ (correction = gain $\times$ surprise)

The correction

- Add the gain times the innovation to the predicted state.
- The correction points toward what the data implies.
- The gain K sets how big that step is (Part 2).

The result

- A new, better estimate that has absorbed the observation.
- The covariance shrinks — you are more certain now.
- This estimate becomes the input to the next predict.

Correction Proportional to Surprise

Bigger surprise, bigger correction

The rule

- The correction is the gain multiplied by the innovation.
- A large surprise moves the estimate a lot.
- A tiny surprise barely moves it at all.

Why this is sensible

- If the data matches the model, leave the estimate alone.
- If the data is far off, pay attention and adjust.
- The gain tunes exactly how much attention to pay.

The Predict-Update Cycle

One pass per observation, forever

Predict, then update, then advance one step and repeat. Each observation triggers exactly one cycle. The estimate and its uncertainty carry from one cycle to the next — no refitting.

The loop

- Predict the state and grow its uncertainty.
- Observe, compute the innovation, and update.
- Advance to the next time step and repeat.

Why it is beautiful

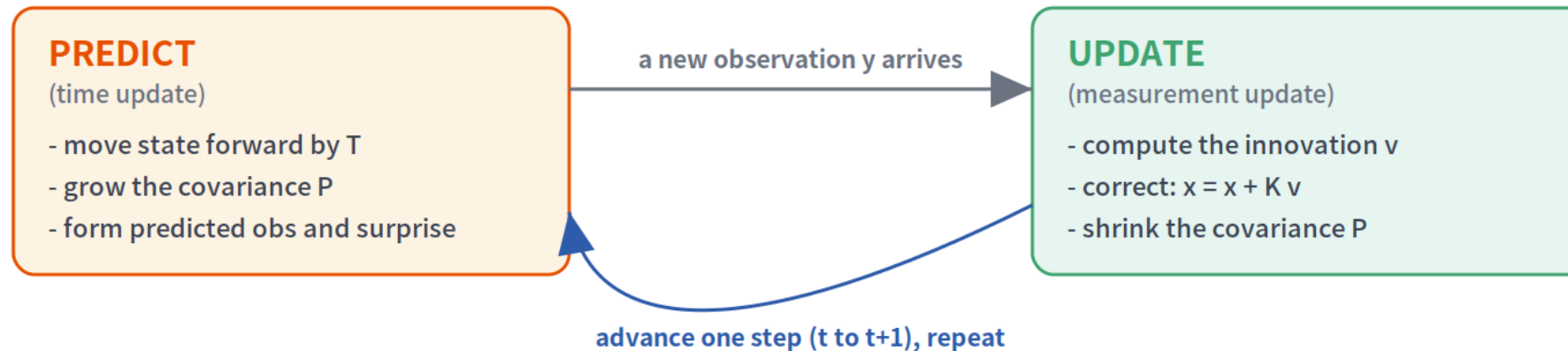
- Constant work per step — it never slows down.
- It handles streaming data naturally.
- The whole past is summarized in two carried objects.

The Predict-Update Cycle

Guess ahead (predict), let a new observation reveal the surprise, correct by the gain times the innovation (update), then advance and repeat. Part 2 makes the correction optimal.

The Predict-Update Cycle

One pass per observation, forever: guess ahead, measure the surprise, correct.



The surprise drives everything

The correction is the Kalman gain K times the innovation v (how wrong the prediction was).

Big surprise or high state uncertainty $\rightarrow$ large correction. Small surprise or precise state $\rightarrow$ barely move.

Part 2 derives the gain K that makes this correction optimal.

The Full Recursion at a Glance

Five equations, two steps - the whole filter on one slide

PREDICT	$\hat{x}_{t t-1} = T \hat{x}_{t-1 t-1}$	$P_{t t-1} = T P_{t-1 t-1} T' + Q$
innovation	$v_t = y_t - Z \hat{x}_{t t-1}$	$F_t = Z P_{t t-1} Z' + H$
gain		
UPDATE	$\hat{x}_{t t} = \hat{x}_{t t-1} + K_t v_t$	$P_{t t} = (I - K_t Z) P_{t t-1}$

Predict (time update)

- Move the state forward by T; inflate its covariance by Q.
- No data is used yet - a pure model forecast.
- You become LESS certain.

Update (measurement update)

- Form the innovation v and its variance F.
- The gain K weighs model vs data; correct by K times v.
- Shrink the covariance - you become MORE certain.

A Single Step, Walked Through

Following one observation through the filter

Predict

- Start from last estimate 10.0 with some uncertainty.
- Model says the level is roughly unchanged: predict 10.0.
- Uncertainty grows a little (process noise added).

Update

- The new observation comes in at 12.0 → surprise = +2.0.
- With a gain of about 0.4, correct by $0.4 \times 2.0 = 0.8$.
- New estimate 10.8, with smaller uncertainty. Repeat.

Filtering in Real Time

Built for streaming data

The recursive advantage

- Each new point costs the same, small amount of work.
- You never re-process the whole history.
- The estimate is always current, the instant data lands.

Where this matters

- Live trading signals and risk estimates.
- High-frequency and streaming data.
- Anywhere decisions can't wait for a batch re-fit.

Initializing the Recursion

Every recursion needs a first guess

The starting point

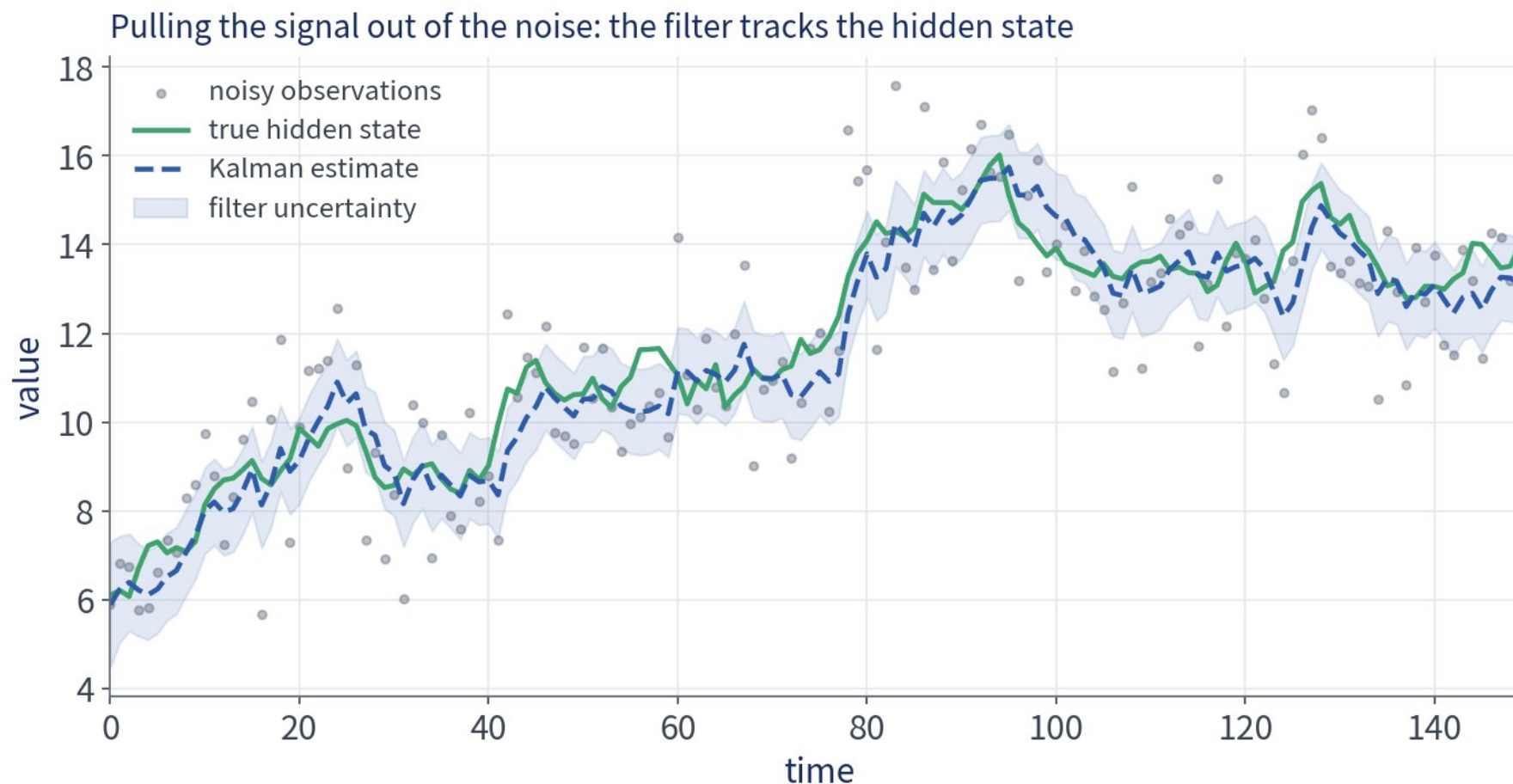
- Begin with a prior estimate and a prior covariance.
- If you know little, start with a large covariance.
- That tells the filter to trust early data heavily.

What happens next

- A vague prior is quickly overwritten by the data.
- The estimate converges as observations accumulate.
- A poor start washes out — the filter self-corrects.

Pulling the Signal Out of the Noise

Noisy observations go in; a clean estimate of the hidden state comes out, with an uncertainty band. This denoising is the filter's most visible payoff.



Recursive vs Batch Estimation

Update, don't re-solve

The batch way

- Re-estimate over the entire dataset each time.
- Cost grows with the history — soon infeasible live.
- Wasteful: most of the answer hasn't changed.

The recursive way

- Carry a compact summary and update it.
- Constant cost per step, forever.
- Mathematically equivalent, but vastly cheaper.

What Parts 2 and 3 Add

From mechanism to mastery

Part 2 — the gain

- The exact formula for the Kalman gain K .
- How it balances trust in the model vs the data.
- The full five-equation recursion, and when it is optimal.

Part 3 — applications

- Denoising and a dynamic, drifting regression beta.
- Pairs trading Stage III: a self-adjusting hedge ratio.
- Practical cautions and the bridge to hidden states (Week 10).

Implement Predict-Update by Hand

Direct Claude Code to build the cycle from scratch

GOAL Code a scalar Kalman filter yourself to feel the two steps — then check it against a library.

[INPUT] Prompt · Claude Code

Write and run `kalman_by_hand.py` that:

1. simulates a local level: a hidden random-walk state plus measurement noise;
 2. implements the filter as an explicit loop with a PREDICT step and an UPDATE step (no library);
 3. at each step prints the innovation and the correction (gain x innovation);
 4. plots the observations, the true state, and the filtered estimate with its uncertainty band;
 5. cross-checks against `statsmodels`' filter.
- Show that predict grows P and update shrinks it.

How to work with it

- Watch P rise in predict, fall in update.
- The innovation should look like white noise.
- Your loop should match the library closely.
- Keep it; Part 2 adds the optimal gain.

Deliverable: `kalman_by_hand.py` + a filtered-state plot and a library cross-check.

Three Questions Before You Move On

If you can answer these, Part 1 has landed

Q1	What happens to the covariance P in the predict step vs the update step?	Grows, then shrinks
Q2	What exactly is the innovation?	Actual minus predicted obs
Q3	In the update, the correction equals what times what?	Gain times innovation

All three come from the two steps, the surprise, and the correction rule.

Three Misconceptions to Avoid

Where Kalman intuition slips

Common myths

- "The filter just smooths the data."
- "It re-processes all past data each step."
- "Predict makes you more certain about the state."

The reality

- It fuses a model with data, weighted by uncertainty.
- It is recursive — only a compact summary is carried.
- Predict makes you LESS certain; only data (update) sharpens it.

Three Things to Carry Forward

Part 1 in one screen

1

Two steps, repeated forever

Predict moves the state forward and grows its uncertainty; update corrects it with new data and shrinks the uncertainty. One cycle per observation, recursively.

2

The innovation drives every correction

The surprise — actual minus predicted observation — is the only new information a data point brings. The update is a correction built entirely from it.

3

The correction is gain times surprise

How much you move depends on the Kalman gain, which weighs the model against the data. Deriving that optimal gain is the subject of Part 2.

Next — Part 2: the Kalman gain, the covariance, the full recursion, and when it is optimal.